Data Structures Using C++ 2E

The Big-O Notation

Algorithm Analysis: The Big-O Notation

- Analyze algorithm after design
- Example
  - 50 packages delivered to 50 different houses
  - 50 houses one mile apart, in the same area

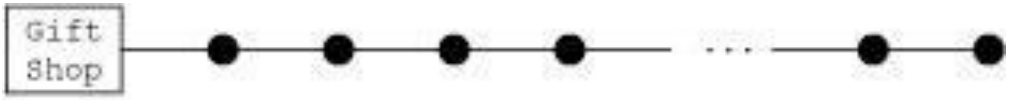


FIGURE 1-1 Gift shop and each dot representing a house

Algorithm Analysis: The Big-O Notation (cont'd.)

- Example (cont'd.)
  - Driver picks up all 50 packages
  - Drives one mile to first house, delivers first package
  - Drives another mile, delivers second package
  - Drives another mile, delivers third package, and so on
  - Distance driven to deliver packages
    - 1+1+1+… +1 = 50 miles
  - Total distance traveled: 50 + 50 = 100 miles

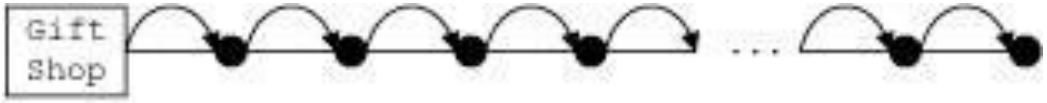


FIGURE 1-2 Package delivering scheme

Algorithm Analysis: The Big-O Notation (cont'd.)

- Example (cont'd.)
  - Similar route to deliver another set of 50 packages
    - Driver picks up first package, drives one mile to the first house, delivers package, returns to the shop
    - Driver picks up second package, drives two miles, delivers second package, returns to the shop
  - Total distance traveled
    - 2 * (1+2+3+…+50) = 2550 miles

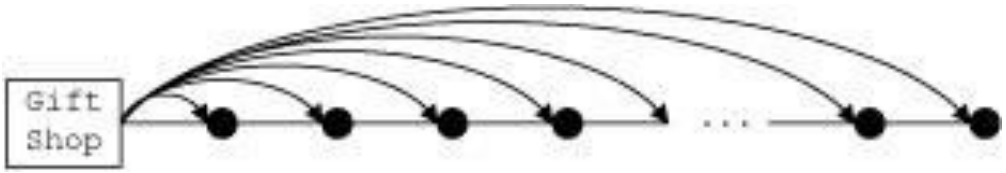


FIGURE 1-3 Another package delivery scheme

Algorithm Analysis: The Big-O Notation (cont'd.)

- Example (cont'd.)
  - *n* packages to deliver to *n* houses, each one mile apart
  - First scheme: total distance traveled
    - $1+1+1+\ldots+n = 2n$ miles
    - Function of n
  - Second scheme: total distance traveled
    - $2 * (1+2+3+\ldots+n) = 2*(n(n+1) / 2) = n^2+n$
    - Function of n^2

Algorithm Analysis: The Big-O Notation (cont'd.)

TABLE 1-1 Various values of n, $2n$, n^2, and $n^2 + n$

n	$2n$	n^2	$n^2 + n$
1	2	1	2
10	20	100	110
100	200	10,000	10,100
1000	2000	1,000,000	1,001,000
10,000	20,000	100,000,000	100,010,000

(n) and (2n) are close, so we magnify (n)
(n^2) and (n^2 + n) are close, so we magnify (n^2)

Algorithm Analysis: The Big-O Notation (cont'd.)

TABLE 1-1 Various values of n, $2n$, n^2, and $n^2 + n$

n	$2n$	n^2	$n^2 + n$
1	2	1	2
10	20	100	110
100	200	10,000	10,100
1000	2000	1,000,000	1,001,000
10,000	20,000	100,000,000	100,010,000

When n becomes too large, n and n^2 becomes very different

Algorithm Analysis: The Big-O Notation (cont'd.)

- Analyzing an algorithm
  - Count number of operations performed
    - Not affected by computer speed

Algorithm Analysis: The Big-O Notation (cont'd.)

- Example 1-1
  - Illustrates fixed number of executed operations

```
cout << "Enter two numbers";                          //Line 1

cin >> num1 >> num2;                                  //Line 2

if (num1 >= num2)                                     //Line 3
    max = num1;                                       //Line 4
else                                                  //Line 5
    max = num2;                                       //Line 6

cout << "The maximum number is: " << max << endl;     //Line 7
```

Only one of them will be executed

1 operation

2 operations

1 operation

1 operation

1 operation

3 operations

Total of 8 operations

Algorithm Analysis: The Big-O Notation

- Example 1-2 Illustrates dominant operations

```
cout << "Enter positive integers ending with -1" << endl;   //Line 1

count = 0;                                                  //Line 2
sum = 0;                                                    //Line 3

cin >> num;                                                 //Line 4

while (num != -1)                                           //Line 5
{
    sum = sum + num;                                        //Line 6
    count++;                                                //Line 7
    cin >> num;                                             //Line 8
}

cout << "The sum of the numbers is: " << sum << endl;       //Line 9

if (count != 0)                                             //Line 10
    average = sum / count;                                  //Line 11
else                                                        //Line 12
    average = 0;                                            //Line 13

cout << "The average is: " << average << endl;              //Line 14
```

2 operations

1 operation

1 operation

1 operation

N+1 operations

2N operations

N operations

N operations

3 operations

1 operation

2 operations

1 operation

3 operation

N times the condition is TRUE + 1 time the condition is FALSE

Executed while the cond. is TRUE

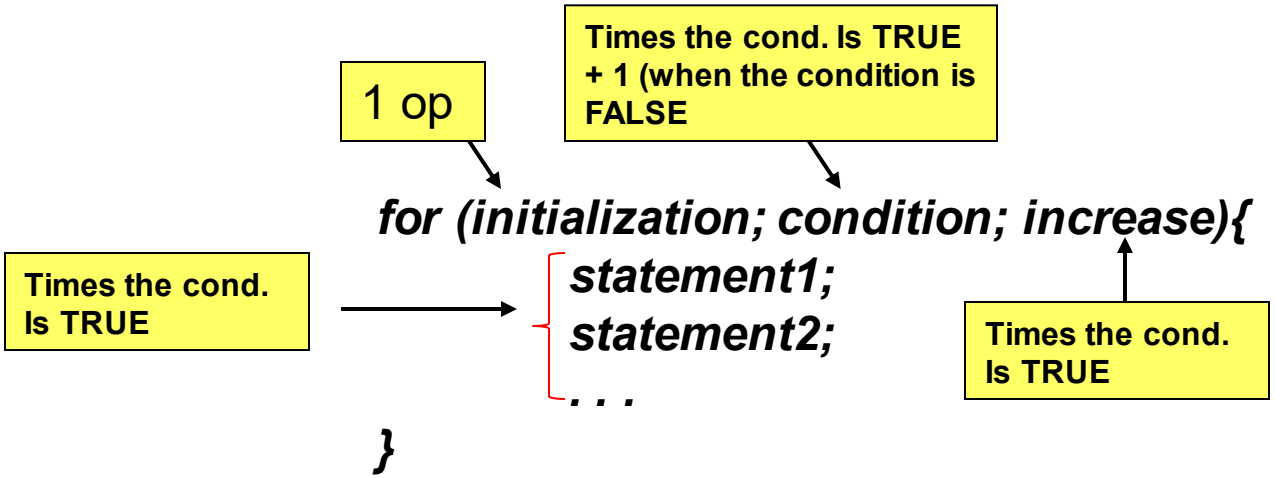
Only one of them will be executed, take the max: 2

If the while loop executes N times then:

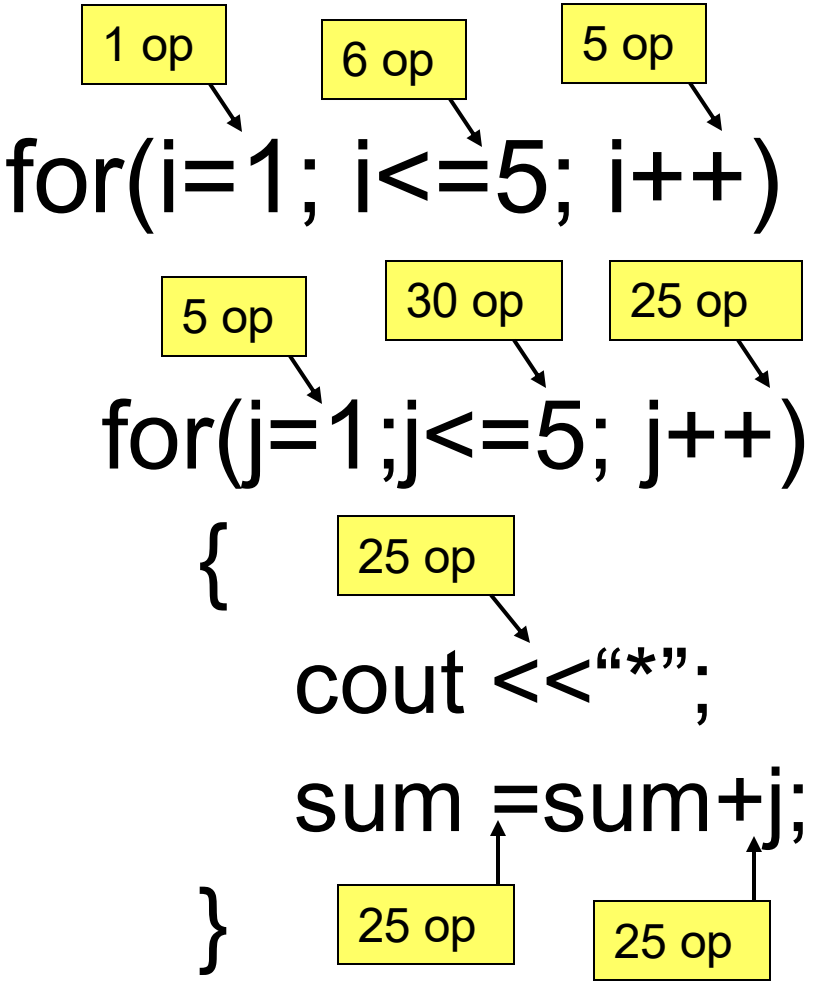
2+1+1+1+5*N + 1 + 3 + 1 + (2) + 3 = 5N+(15)

Algorithm Analysis: The Big-O Notation

How to count *for* loops



Example



Finally: 147

```
for(i=1; i<=n; i++)
    for(j=1; j<=n; j++){
        cout <<“*”;
        Sum=sum + j;
        }
```

$2n+2+$
$2n^2+2n+$
$3n^2$

$= 5n^2+ 4n+2$

Algorithm Analysis: The Big-O Notation (cont'd.)

TABLE 1-2 Growth rates of various functions

n	$\log_2 n$	$n \log_2 n$	n^2	2^n
1	0	0	1	2
2	1	2	2	4
4	2	8	16	16
8	3	24	64	256
16	4	64	256	65,536
32	5	160	1024	4,294,967,296

Algorithm Analysis: The Big-O Notation (cont'd.)

TABLE 1-3 Time for $f(n)$ instructions on a computer that executes 1 billion instructions per second (1 GHz)

n	$f(n) = n$	$f(n) = \log_2 n$	$f(n) = n\log_2 n$	$f(n) = n^2$	$f(n) = 2^n$
10	0.01µs	0.003µs	0.033µs	0.1µs	1µs
20	0.02µs	0.004µs	0.086µs	0.4µs	1ms
30	0.03µs	0.005µs	0.147µs	0.9µs	1s
40	0.04µs	0.005µs	0.213µs	1.6µs	18.3min
50	0.05µs	0.006µs	0.282µs	2.5µs	13 days
100	0.10µs	0.007µs	0.664µs	10µs	4×10^{13} years
1000	1.00µs	0.010µs	9.966µs	1ms	
10,000	10µs	0.013µs	130µs	100ms	
100,000	0.10ms	0.017µs	1.67ms	10s	
1,000,000	1 ms	0.020µs	19.93ms	16.7m	
10,000,000	0.01s	0.023µs	0.23s	1.16 days	
100,000,000	0.10s	0.027µs	2.66s	115.7 days	

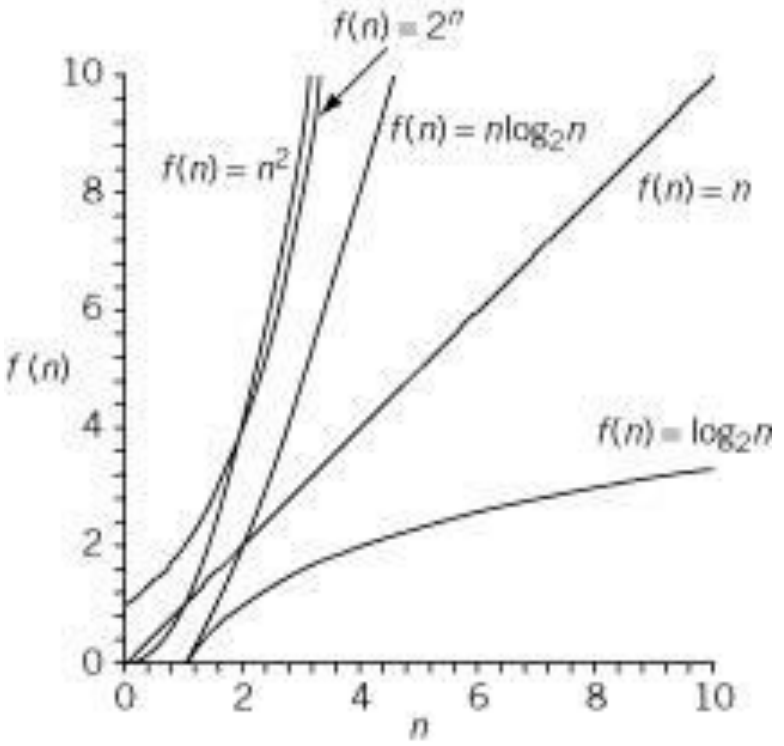


Figure 1-4 Growth rate of functions in Table 1-3

Algorithm Analysis: The Big-O Notation (cont'd.)

- Notation useful in describing algorithm behavior
  - Shows how a function *f(n)* grows as *n* increases without bound
- Asymptotic
  - Study of the function *f* as *n* becomes larger and larger without bound
  - Examples of functions
    - $g(n)=n^2$ (no linear term)
    - $f(n)=n^2 + 4n + 20$

Algorithm Analysis: The Big-O Notation (cont'd.)

- As n becomes larger and larger
  - Term $4n + 20$ in $f(n)$ becomes insignificant
  - Term n^2 becomes dominant term

TABLE 1-4 Growth rate of n^2 and $n^2 + 4n + 20n$

n	$g(n) = n^2$	$f(n) = n^2 + 4n + 20$
10	100	160
50	2500	2720
100	10,000	10,420
1000	1,000,000	1,004,020
10,000	100,000,000	100,040,020

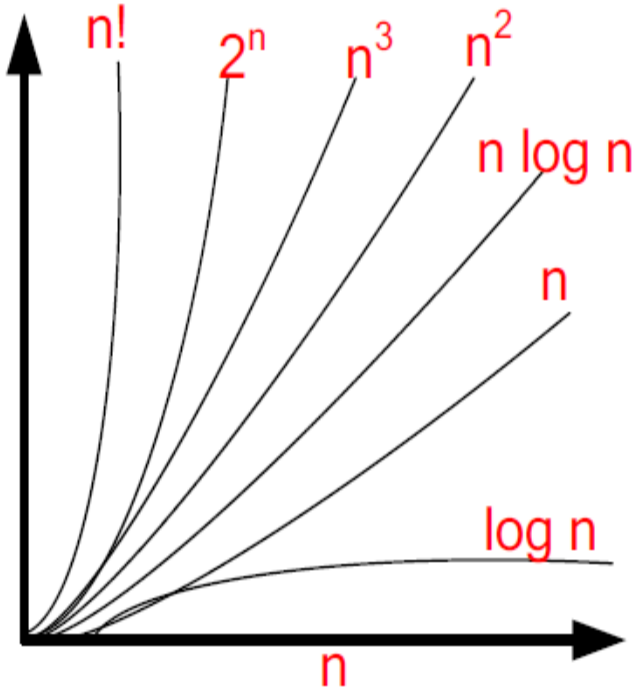
Algorithm Analysis: The Big-O Notation (cont'd.)

- Algorithm analysis
  - If function complexity can be described by complexity of a quadratic function without the linear term
    - We say the function is of $O(n^2)$ or Big-O *of* n^2
- Let *f* and *g* be real-valued functions
  - Assume *f* and *g* nonnegative
    - For all real numbers *n*, $f(n) >= 0$ and $g(n) >= 0$
- $f(n)$ is Big-O of $g(n)$: written $f(n) = O(g(n))$
  - If there exists positive constants *c* and n_0 such that $f(n) <= cg(n)$ for all $n >= n_0$

Algorithm Analysis: The Big-O Notation (cont'd.)

TABLE 1-5 Some Big-O functions that appear in algorithm analysis

Function $g(n)$	Growth rate of $f(n)$
$g(n) = 1$	The growth rate is constant and so does not depend on n, the size of the problem.
$g(n) = \log_2 n$	The growth rate is a function of $\log_2 n$. Because a logarithm function grows slowly, the growth rate of the function f is also slow.
$g(n) = n$	The growth rate is linear. The growth rate of f is directly proportional to the size of the problem.
$g(n) = n\log_2 n$	The growth rate is faster than the linear algorithm.
$g(n) = n^2$	The growth rate of such functions increases rapidly with the size of the problem. The growth rate is quadrupled when the problem size is doubled.
$g(n) = 2^n$	The growth rate is exponential. The growth rate is squared when the problem size is doubled.



Answer:

1) $6 \log n + 9 n = O(n)$

2) $5 n^2 + n^{3/2} = O(n^2)$

3) $5 n^{5/2} + n^{2/5} = O(n^{2.5})$

4) $3 n^4 + n \log n = O(n^4)$

Q1 Find out the big-O notation for the following expressions

F(n)	O(g(n))
$2^6 + n^2 Logn^3 + n^{1.6} + 100n^4$	$O(n^4)$
$2^{10} + 3^5$	$O(1)$
$nLogn + 15n + .5n^2$	$O(n^2)$
$1000n^2 + 16n + 2^n$	$O(2^n)$
$nLog2^n + n^2$	$O(n^2)$

Find the Big-O notation for the following expressions.

F(n)	O(g(n))
$10n + 5$	O(n)
190	O(1)
$3n^2 + 4n + 1$	$O(n^2)$
$n^5 + n^3 + 3n$	$O(n^5)$
$5n^2(n+3)$	$O(n^3)$
$n(n-1)$	$O(n^2)$
$3n * \log(n) + 11$	O(nlogn)
$2n + 300 + n\log n + n^2$	$O(n^2)$

Q2. (12 pts) Fill each box of the boxes shown below with the number of basic operations done by the underlined statement pointed to by the arrow attached to the box.